Front-end & Back-end

Marc Muntané, Pol Carvajal,
Bryan Joya, Joan Rodrigues
STUCOM DAW 2

Index

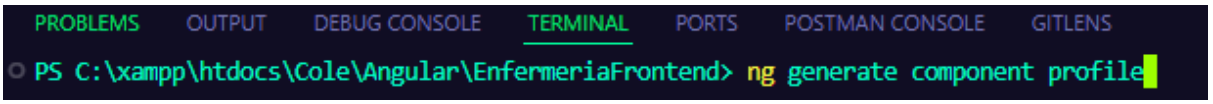
Profile **3**
Create ‘Profile’ 3
Load ‘Profile’ 5
Update ‘Profile 6
Frontend TypeScript Programming: 6
Injectable Frontend programming: 9
HTML Frontend Programming: 10
Backend Programming: 11
Delete ‘Profile’ 12
Frontend TypeScript and HTML Programming: 12
Injectable Frontend Programming and Backend 13
Endpoint ‘searchByName’ **14**
Frontend TypeScript Programming: 14
Injectable Frontend programming: 15
HTML Frontend Programming: 16
Backend Programming: 17
Endpoint ‘Register’ **18**
Frontend TypeScript Programming: 18
Injectable Frontend programming: 19
HTML Frontend Programming: 20
Backend Programming: 21
Endpoint ‘Login’ **22**
Frontend TypeScript Programming: 22
Injectable Frontend programming and Backend 23
HTML Frontend Programming: 24
Nurse list ‘getAll’ **25**
Frontend TypeScript Programming: 25
Injectable Frontend programming: 26
HTML Frontend Programming: 26
Backend Programming: 27

Profile

Create 'Profile'

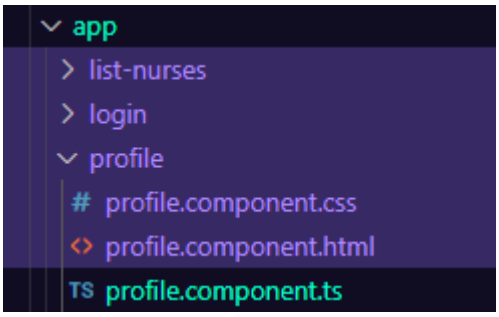
To create a component can go to `Terminal` and execute the next command:

```
ng generate component profile
```



This command creates a folder with 3 files.

- The `HTML`, the `front-end` part.
- Next, the `CSS`, the visual part of `HTML`.
- And finally the `Type Script` part, the `back-end`.



To begin, I created the user icon using `HTLM` and `CSS`, allowing users to access their profile.

When clicking on the profile icon, a box appears displaying the `user's first name, last name, and username`. Within the same box, users can also update their profile and change their password.

When clicking on '`Change Password`,' a new box appears visually, allowing the user to update their `password`.

Load 'Profile'

Upon initialization (`ngOnInit`), the user's profile data is retrieved from the backend using `this.enfermerosService.getProfile(id)`. The ID is obtained from `localStorage`, and the response is processed to populate the form fields. If data is found, the properties `name`, `surname`, `user`, and `pass` are assigned to the `enfermero` object; otherwise, error messages are logged in the console.

```
ngOnInit(): void {
  let id = parseInt(localStorage.getItem('id') || '');
  this.enfermerosService.getProfile(id).subscribe({
    next: (result) => {
      if (Array.isArray(result) && result.length > 0) {
        this.enfermero = {
          nif: result[0]?.dni || '',
          name: result[0]?.name || '',
          surname: result[0]?.surname || '',
          user: result[0]?.user || '',
          pass: result[0]?.pass || '',
        };
      } else {
        console.error('No se encontraron datos del enfermero.');
      }
    },
    error: (error) => {
      console.error('Error al obtener los datos:', error);
    },
  });
}
```

The profile update form is structured using Angular's reactive forms. It includes fields for the user's first name, last name, and username. Each input field is bound to the corresponding model property using `[(ngModel)]` and `formControlName`, ensuring that changes are dynamically reflected in the component.

```
<div [formGroup]="fromUpdateProfile">
  <div class="name">
    <label>
      Nombre:
      <input type="text" id="input2" class="input2" [(ngModel)]="enfermero.name" formControlName="name" />
    </label>
  </div>
  <div class="surname">
    <label>
      Apellido:
      <input type="text" id="input3" class="input3" [(ngModel)]="enfermero.surname" formControlName="surname" />
    </label>
  </div>
  <div class="user">
    <label>
      Usuario:
      <input type="text" id="input4" class="input4" [(ngModel)]="enfermero.user" formControlName="user" />
    </label>
  </div>
```

A backend endpoint (`/Nurses/SearchId/`) is implemented using Symfony. This method (`findById`) retrieves a nurse's profile based on an ID provided as a query parameter. The repository fetches the corresponding record from the database. If a nurse is found, their details (`DNI, name, surname, username, and password`) are returned as a JSON response. If no record is found, an error message is sent back to the client.

```
#[Route(path: '/Nurses/SearchId/', name: 'app_nurse_search_id', methods: ['GET'])]

6 references | 0 overrides
public function findById(Request $request, EntityManagerInterface $entityManager): JsonResponse
{
    $id = $request->query->get(key: 'id');

    $nursesRepository = $entityManager->getRepository(className: Nurses::class);

    $nurses = $nursesRepository->find(id: $id);

    if ($nurses !== null) {
        $result[] = [
            'dni' => $nurses->getDni(),
            'name' => $nurses->getName(),
            'surname' => $nurses->getSurname(),
            'user' => $nurses->getUser(),
            'pass' => $nurses->getPassword()
        ];

        return new JsonResponse(data: $result);
    } else {
        return new JsonResponse(data: 'No se encontro ningun enfermero.');
    }
}
```

Update ‘Profile

Frontend TypeScript Programming:

This function, `edicion()`, allows editing the input fields in a form:

1. It selects three input fields (`input2`, `input3`, and `input4`) by their IDs.

2. It enables these fields by setting their `disabled` property to `false`, making them editable.
3. It sets a flag `this.isEditing` to `true`, indicating the form is now in editing mode.

```
edicion() {
  let name = document.getElementById('input2') as HTMLInputElement | null;
  let surname = document.getElementById('input3') as HTMLInputElement | null;
  let user = document.getElementById('input4') as HTMLInputElement | null;
  name!.disabled = false;
  surname!.disabled = false;
  user!.disabled = false;
  this.isEditing = true;
}
```

This code creates a **reactive form** with three fields: `name`, `surname`, and `user`. Each field is initialized with an empty value and set to **disabled**, meaning the user can't edit them. The fields also have a **required** validation, so they must be filled if they were enabled. This form is likely used for displaying the nurse's data in a non-editable way.

```
  this.fromUpdateProfile = new FormGroup({
    name: new FormControl({ value: '', disabled: true }, Validators.required),
    surname: new FormControl(
      { value: '', disabled: true },
      Validators.required
    ),
    user: new FormControl({ value: '', disabled: true }, Validators.required),
  });
}
```

The `updateUser` function updates a nurse's profile by first retrieving the `id` from **local storage** and collecting the nurse's details (`nif`, `name`, `surname`, and `user`) from a form. It then checks if the password is being changed—if not, it keeps the current password; otherwise, it takes the new one from a password change form. After gathering all the data, it calls the `updateProfile` function from `enfermerosService` to send the updated information to the server. If the update is successful, it displays a success message; if there's an error, it logs it to the console.

```
updateUser() {
  const enfermero = {
    id: parseInt(localStorage.getItem('id') || ''),
    nif: this.enfermero.nif,
    name: this.fromUpdateProfile.get('name')?.value,
    surname: this.fromUpdateProfile.get('surname')?.value,
    user: this.fromUpdateProfile.get('user')?.value,
  };

  let password;
  if (!this.isChangePassword) {
    password = this.enfermero.pass;
  } else {
    password = this.changePasswordForm.get('newPassword')?.value;
  }
  this.enfermerosService
    .updateProfile(
      enfermero.id,
      enfermero.nif,
      enfermero.name,
      enfermero.surname,
      enfermero.user,
      password
    )
    .subscribe({
      next: (result) => {
        this.MessageShowed(
          'El enfermero se ha actualizado correctamente',
          'success'
        );
      },
      error: (error) => {
        console.error('Error al obtener los datos:', error);
      },
    });
}
```

This code initializes a **reactive form** (`fromUpdateProfile`) with three fields: `name`, `surname`, and `user`. Each field is a **FormControl** with an initial empty value and is set to **disabled** (making them uneditable). The fields are also marked as **required**, meaning they need to be filled if they were enabled. This form is likely used to display information without allowing the user to modify it directly.

```
this.changePasswordForm = new FormGroup({
  oldPassword: new FormControl('', [Validators.required]),
  newPassword: new FormControl('', [Validators.required]),
  confirmPassword: new FormControl('', [Validators.required]),
});
```

`openChangePasswordModal()`: This function sets the variable `isChangePasswordModalOpen` to `true`, which likely opens a modal window allowing the user to change their password.
`closeChangePasswordModal()`: This function sets `isChangePasswordModalOpen` to `false`, which likely closes the password change modal.

```
openChangePasswordModal() {
  this.isChangePasswordModalOpen = true;
}

closeChangePasswordModal() {
  this.isChangePasswordModalOpen = false;
}
```

The function checks if the password change form is valid, retrieves the old, new, and confirm passwords, and verifies the old password matches the current one. If the new password and confirm password don't match, it shows an error. If they match, it shows a success message and closes the modal. If the old password is wrong or the form is invalid, it shows an error message.

```
saveNewPassword() {
  if (this.changePasswordForm.valid) {
    const oldPassword = this.changePasswordForm.get('oldPassword')?.value;
    const newPassword = this.changePasswordForm.get('newPassword')?.value;
    const confirmPassword =
      this.changePasswordForm.get('confirmPassword')?.value;

    if (oldPassword == this.enfermero.pass) {
      if (newPassword !== confirmPassword) {
        this.MessageShowed('Las contraseñas nuevas no coinciden', 'error');
        return;
      }
      this.MessageShowed('Contraseña actualizada con éxito', 'success');
      this.closeChangePasswordModal();
    } else {
      this.MessageShowed(
        'Las contraseña introducida antigua no coincide',
        'error'
      );
    }
  } else {
    this.MessageShowed('Por favor, completa todos los campos', 'error');
  }
}
```

Injectable Frontend programming:

The `updateProfile` function in TypeScript updates a nurse's profile by sending an **HTTP PUT** request to a server. It takes the `id` and other details like `dni`, `name`, `surname`, `user`, and `password`, packages them into an object, and sends them to the URL `/Nurses/{id}`. It also sets **HTTP headers** to specify that the data is in JSON format. Finally, it returns an **Observable**, allowing the response to be handled asynchronously in an **Angular** application.

```
updateProfile(id: number, dni: string, name: string, surname: string, user: string, password: string): Observable<any>
  let url = `/Nurses/${id}`;

  const body = {
      dni: dni,
      name: name,
      surname: surname,
      user: user,
      password: password
  };

  const headers = new HttpHeaders({ 'Content-Type': 'application/json' });

  return this.http.put<any>(url, body, { headers });
}
```

HTML Frontend Programming:

This HTML code is for a profile form where a nurse can view and edit their details. It uses **Angular's reactive forms** (`[formGroup]` and `formControlName`) for binding form fields like `name`, `surname`, and `user` to the `fromUpdateProfile` form. The `[(ngModel)]` directive is used for two-way data binding with the `enfermero` object.

- The **"Cambiar contraseña"** link triggers the `openChangePasswordModal()` function to open a password change modal.
- The form has two buttons:
  - The **"Modificar"** button is shown when `isEditing` is false, triggering the `edicion()` function to allow editing.
  - The **"Guardar"** button is shown when `isEditing` is true, triggering the `updateUser()` function to save the changes.

```
<section>
  <div class="login">
    <form class="form">
      <div class="inputs">
        <label class="title"> Profile </label>
        <div [formGroup]="fromUpdateProfile">
          <div class="name">
            <label>
              Nombre:
              <input type="text" id="input2" class="input2" [(ngModel)]="enfermero.name" formControlName="name" />
            </label>
          </div>
          <div class="surname">
            <label>
              Apellido:
              <input type="text" id="input3" class="input3" [(ngModel)]="enfermero.surname" formControlName="surname" />
            </label>
          </div>
          <div class="user">
            <label>
              Usuario:
              <input type="text" id="input4" class="input4" [(ngModel)]="enfermero.user" formControlName="user" />
            </label>
          </div>

          <div class="pass">
            <a class="change-password-link" (click)="openChangePasswordModal()">
              Cambiar contraseña
            </a>
          </div>
          <div style="display: flex;">
            <button *ngIf="!isEditing" type="button" class="submit" (click)="edicion()">
              Modificar
            </button>
            <button *ngIf="isEditing" type="submit" class="submit" (click)="updateUser()">
              Guardar
            </button>
          </div>
```

This HTML code represents a **password change modal** that appears when `isChangePasswordModalOpen` is `true`.
It includes input fields for the **old password**, **new password**, and **confirm password**, bound to the `changePasswordForm`.
The **"Guardar"** button triggers the `saveNewPassword()` function to save the new password.
The **"Cerrar"** button calls the `closeChangePasswordModal()` function to close the modal.

```
    <div *ngIf="isChangePasswordModalOpen" class="modal-overlay">
      <div class="modal-content">
        <label class="title"> contraseña </label>
        <div class="passwords" [formGroup]="changePasswordForm">
          <label>
            Antigua contraseña:
            <input type="password1" formControlName="oldPassword" />
          </label>
          <label>
            Nueva contraseña:
            <input type="password2" formControlName="newPassword" />
          </label>
          <label>
            Repetir contraseña:
            <input type="password3" formControlName="confirmPassword" />
          </label>
        </div>
        <button class="safe-modal" (click)="saveNewPassword()">Guardar</button>
        <button class="close-modal" (click)="closeChangePasswordModal()">Cerrar</button>
      </div>
    </div>
  </form>
</div>
```

Backend Programming:

The function listens for a **PUT request** at `/Nurses/{id}` to update a nurse's information.
It retrieves the nurse from the database using the `id` provided in the URL.
The updated data (like `dni`, `name`, `surname`, `user`, `password`) is extracted from the request body.
If the nurse is found, their information is updated and saved using the entity manager.
A **202 response** is returned with the updated data if successful.
If the nurse is not found, a **404 error** is returned.

```
    #[Route('/Nurses/{id}', name: 'app_nurse_update', methods: ['PUT'])]
    6 references | 0 overrides
    public function updateNurse(EntityManagerInterface $entityManager, int $id, Request $request): JsonResponse
    {
        $enfermeroRepository = $entityManager->getRepository(className: Nurses::class);
        $enfermero = $enfermeroRepository->find($id);
        if ($enfermero) {
            $data = json_decode(json: $request->getContent(), associative: true);
            $enfermero->setDni($data['dni']);
            $enfermero->setName($data['name']);
            $enfermero->setSurname($data['surname']);
            $enfermero->setUser($data['user']);
            $enfermero->setPassword($data['password']);
            $entityManager->flush();
            return new JsonResponse(data: $data, status: 202);
        } else {
            return new JsonResponse(data: 'No se encontro ningun enfermero.', status: 404);
        }
    }
}
```

Delete 'Profile'

Frontend TypeScript and HTML Programming:

```
deleteUser() {
  this.enfermerosService.delete(this.storage.getItem('id')).subscribe(
    (data) => {
      if (!data) {
        return this.MessageShowedDelete(
          'No existe el id de este usuario.',
          'error'
        );
      } else {
        this.closeDelete();

        this.MessageShowedDelete(
          'Usuario eliminado correctamente',
          'success'
        );
      }
    }
  );
}
```

The main function to delete the user in the .ts file is the deleteUser() function. In this function, we call the function found in the injectable which is responsible for connecting to the backend.
The backend will return true if the user has been deleted correctly, and false if there has been any type of error that has prevented the deletion of said user.

```
<div *ngIf="deletePushed" class="modal-overlay style">
  <div class="modal-content">
    <h3>Estas seguro de querer eliminar este usuario</h3>
    <p>(No se podra recuperar)</p>
    <button class="submit delete2" (click)="deleteUser()">Eliminar</button>
    <button class="submit" (click)="closeDelete()">Cancelar</button>
  </div>
</div>
```

Other functions created in this file are closeDelete() and openDelete(). These two functions allow us that when we want to delete a user, and we press the delete button, a pop-up window opens that asks the user if they are sure they want to delete it, if they say yes, the pop-up window will be closed using the closeDelete() and deleted with the deleteUser() function will be executed, which will delete the user and log them out. If, instead, you

indicate that you do not want to delete your user, the pop-up window will simply close, allowing the user to continue browsing the application.

Injectable Frontend Programming and Backend

```
delete(id: number): Observable<Object> {
  let url = `/Nurses/${encodeURIComponent(
    id
  )}`;
  const headers = new HttpHeaders({ 'Content-Type': 'application/json' });

  return this.http.delete(url, { headers });
}
```

Front End Function

```
#[Route(path: '/Nurses/{id}', name: 'app_nurses_delete', methods: ['DELETE'])]
8 references | 0 overrides
public function deleteByID(EntityManagerInterface $entityManager, Int $id): JsonResponse
{
    $enfermeroRepository = $entityManager->getRepository(class: Nurses::class);

    $enfermero = $enfermeroRepository->find(id: $id);

    if (!$enfermero) {
        return new JsonResponse(data: false);
    } else {
        $entityManager->remove(object: $enfermero);
        $entityManager->flush();
        return new JsonResponse(data: true);
    }
}
```

Back End Endpoint

In the injectable file, we will create a function called delete(id: number), which receives as an input parameter the "id" of the user to be deleted.
To do this, we must perform a delete type http query, in which, indicating in its url, the id of the user to be deleted, in the backend, it will use the by function to find the user that agrees with this "id" If it finds a user with this id, it will delete it and return true to the frontend, while if it does not find any user with that "id" it will return false.

Endpoint 'searchByName'

Frontend TypeScript Programming:

```
BuscarPorNombre(): void {
  const inputElement = document.getElementById(
    'nombre'
  ) as HTMLInputElement | null;

  if (inputElement) {
    const inputNombre = inputElement.value.trim();

    if (this.resultados ≠ null) {
      this.resultados = [];
    }

    if (inputNombre === '') {
      console.log('Introduce un nombre o usuario para buscar.');
      return;
    }

    this.enfermerosService.getNursesSearchName(inputNombre).subscribe({
      next: (response) => {
        if (response.data && Array.isArray(response.data)) {
          this.resultados = response.data.filter(
            (enfermero: any) =>
              enfermero.user.toUpperCase() === inputNombre.toUpperCase() ||
              enfermero.name.toUpperCase() === inputNombre.toUpperCase()
          );
          console.log('Resultados encontrados:', this.resultados);
        } else {
          console.log(
            'No se encontró ningún enfermero con ese nombre o usuario.'
          );
          this.resultados = [];
        }
      },
      error: (error) => {
        console.error('Error al buscar enfermeros:', error);
      },
    });
  }
}
```

This code searches for nurses by name or username. It calls `getNursesSearchName(inputNombre)`, and when it gets a response:

- If the data is valid, it filters the nurses to find those whose `user` or `name` matches `inputNombre` (ignoring case).
- The results are saved in `this.resultados` and logged in the console.
- If no match is found, it clears `this.resultados` and logs a message.
- If there's an error, it logs it to the console.

Injectable Frontend programming:

```
getNursesSearchName(
  value: string
): Observable<{ message: string; data: any[] }> {
  const url = '/Nurses/SearchName?name=${encodeURIComponent(
    value
  )}&user=${encodeURIComponent(value)}';
  const headers = new HttpHeaders({ 'Content-Type': 'application/json' });

  return this.http.get<{ message: string; data: any[] }>(url, { headers });
}
```

This function searches for nurses by name or username. It:

- Builds a URL with the provided `value`, encoding it to avoid issues.
- Sets the request headers to use JSON.
- Makes an HTTP GET request to fetch nurses matching the `value`.
- Returns an observable with the response, which includes a `message` and a `data` array.

HTML Frontend Programming:

```html
<section class="container-general">
  <div class="titulo">
    <h1>Buscar Por Nombre o Usuario</h1>
  </div>
  <div class="input">
    <input
      style="text-align: center"
      id="nombre"
      type="text"
      name="nombre"
      placeholder="Introduce un nombre o usuario"
      required
      (keyup)="BuscarPorNombre()"
    />
  </div>

  <div class="resultados">
    <div *ngIf="resultados && resultados.length > 0">
      <span *ngFor="let enfermero of resultados">
        Nombre: {{ enfermero.name }} {{ enfermero.surname }} <br />
        Usuario: {{ enfermero.user }} <br />
        <br />
      </span>
    </div>
    <div *ngIf="resultados && resultados.length === 0">
      <p>No se encontraron resultados.</p>
    </div>
  </div>
</section>
```

This HTML displays the search results for nurses:

- If `resultados` has data, it loops through each nurse and shows their name, surname, and username.
- If `resultados` is empty, it displays a message saying "No results found."

Backend Programming:

```php
#[Route(path: '/Nurses/SearchName', name: 'app_nurse_search_name', methods: ['GET'])]
8 references | 0 overrides
public function findByName(Request $request, EntityManagerInterface $entityManager): JsonResponse
{
    $name = $request->query->get(key: 'name');
    $user = $request->query->get(key: 'user');

    if (empty($name) && empty($user)) {
        return new JsonResponse(data: ['message' => 'Por favor, proporciona al menos un criterio de búsqueda.'], status: 400);
    }

    $qb = $entityManager->createQueryBuilder();
    $qb->select(select: 'n')->from(from: Nurses::class, alias: 'n');

    if (!empty($name) || !empty($user)) {
        $searchValue = !empty($name) ? $name : $user;
        $qb->andWhere(where: 'n.name = :searchValue OR n.user = :searchValue')->setParameter(key: 'searchValue', value: $searchValue);
    }

    $query = $qb->getQuery();
    $nurses = $query->getResult();

    if (!empty($nurses)) {
        $result = [];
        foreach ($nurses as $nurse) {
            $result[] = [
                'name' => $nurse->getName(),
                'surname' => $nurse->getSurname(),
                'user' => $nurse->getUser(),
            ];
        }

        return new JsonResponse(data: ['message' => 'Enfermeros encontrados', 'data' => $result]);
    } else {
        return new JsonResponse(data: ['message' => 'No se encontró ningún enfermero.']);
    }
}
```

This PHP function searches for nurses by name or username and returns the results as a JSON response.

- It listens for `GET` requests at `/Nurses/SearchName`.
- It retrieves `name` and `user` from the query parameters.
- If both are empty, it returns a 400 error with a message asking for at least one search criterion.
- It builds a database query using Doctrine's QueryBuilder:
    - If `name` or `user` is provided, it searches for a nurse where `name` or `user` matches the value.
- Executes the query and gets the results.
- If nurses are found, it formats the data into an array and returns a JSON response with the nurses' name, surname, and username.
- If no matches are found, it returns a message saying no nurses were found.

Endpoint ‘Register’

Frontend TypeScript Programming:

```typescript
export class Register {
  registerForm!: FormGroup;

  onRegister() {}

  constructor(
    private enfermeroService: EnfermerosService,
    private router: Router
  ) {
    this.registerForm = new FormGroup({
      nif: new FormControl('', [
        Validators.required,
        Validators.minLength(9),
        Validators.maxLength(9),
      ]),
      name: new FormControl('', [Validators.required]),
      surname: new FormControl('', [Validators.required]),
      user: new FormControl('', [Validators.required]),
      pass: new FormControl('', [Validators.required, Validators.minLength(7)]),
    });
  }

  onSubmit() {
    if (this.registerForm.valid) {
      const enfermero = {
        nif: this.registerForm.get('nif')?.value,
        name: this.registerForm.get('name')?.value,
        surname: this.registerForm.get('surname')?.value,
        user: this.registerForm.get('user')?.value,
        pass: this.registerForm.get('pass')?.value,
      };

      this.enfermeroService
        .setNursesRegister(
          enfermero.nif,
          enfermero.name,
          enfermero.surname,
          enfermero.user,
          enfermero.pass
        )
        .subscribe(
          (response) => {
            this.MessageShowed(
              'Se ha registrado su usuario correctamente',
              'success'
            );
            console.log(response);
          },
          (error) => {
            this.MessageShowed('No se ha registrado su usuario', 'error');
            console.log(error);
          }
        );
    }
  }

  message = '';
  messageType: 'success' | 'error' = 'success';
  showMessage = false;
```

```typescript
  private MessageShowed(message: string, type: 'success' | 'error'): void {
    this.message = message;
    this.messageType = type;
    this.showMessage = true;
    setTimeout(() => {
      this.showMessage = false;
      if (this.messageType == 'success') {
        this.router.navigate(['/Login']);
      }
    }, 2000);
  }
}
```

This class is used to register nurses in an Angular app.

How it works:

- **Setting up the form:**
  - The form (`registerForm`) has fields: `nif`, `name`, `surname`, `user`, and `pass`.
  - Each field has rules, like required and minimum/maximum length.
- **When clicking submit (`onSubmit` method):**
  - It checks if the form is valid.
  - If valid, it collects the form data and sends it to the `enfermeroService`.
  - If the registration is successful, it shows a success message and logs the response.
  - If there's an error, it shows an error message and logs the error.
- **Showing messages (`MessageShowed` method):**
  - It displays success or error messages.
  - After 2 seconds, the message disappears.
  - If registration is successful, it redirects to the login page.

Injectable Frontend programming:

```
setNursesRegister(
  nif: string,
  name: string,
  surname: string,
  user: string,
  pass: string
): Observable<any> {
  let url = '/Nurses/Create';

  const params = new HttpParams()
    .set('dni', nif)
    .set('name', name)
    .set('surname', surname)
    .set('user', user)
    .set('password', pass);

  const headers = new HttpHeaders({ 'Content-Type': 'application/json' });

  return this.http.post<any>(url, null, { headers, params });
}
```

This function sends a request to register a new nurse.

How it works:

- It sets the API URL to `/Nurses/Create`.
- It creates request parameters with the nurse's `nif`, `name`, `surname`, `user`, and `password`.
- It sets the request headers to `Content-Type: application/json`.
- It makes an HTTP `POST` request with the parameters.
- It returns an `Observable` that will handle the response from the server.

HTML Frontend Programming:

```html
<section>
  <div class="login">
    <form [formGroup]="registerForm" (ngSubmit)="onSubmit()" class="form">
      <div class="inputs">
        <label class="title"> Register </label>
        <div class="nif">
          <label>
            DNI:
            <input type="text" formControlName="nif" class="input1" />
          </label>
          <div
            *ngIf="
              registerForm.get('nif')?.invalid &&
              registerForm.get('nif')?.touched
            "
            style="color: red; font-weight: bold"
          >
            El nif es obligatorio.
          </div>
        </div>
        <div class="name">
          <label>
            Nombre:
            <input type="text" formControlName="name" class="input2" />
          </label>
          <div
            *ngIf="
              registerForm.get('name')?.invalid &&
              registerForm.get('name')?.touched
            "
            style="color: red; font-weight: bold"
          >
            El nombre es obligatorio.
          </div>
        </div>
        <div class="surname">
          <label>
            Apellido:
            <input type="text" formControlName="surname" class="input3" />
          </label>
          <div
            *ngIf="
              registerForm.get('surname')?.invalid &&
              registerForm.get('surname')?.touched
            "
            style="color: red; font-weight: bold"
          >
            El apellido es obligatorio.
          </div>
        </div>
```

```html
        <div class="user">
          <label>
            Usuario:
            <input type="text" formControlName="user" class="input4" />
          </label>
          <div
            *ngIf="
              registerForm.get('user')?.invalid &&
              registerForm.get('user')?.touched
            "
            style="color: red; font-weight: bold"
          >
            El usuario es obligatorio.
          </div>
        </div>
        <div class="pass">
          <label>
            Contraseña:
            <input type="password" formControlName="pass" class="" />
          </label>
          <div
            *ngIf="
              registerForm.get('pass')?.invalid &&
              registerForm.get('pass')?.touched
            "
            style="color: red; font-weight: bold"
          >
            La contraseña es obligatoria.
          </div>
        </div>
      </div>
      <button type="submit" [disabled]="registerForm.invalid" class="submit">
        Access
      </button>
    </form>
  </div>
  <div
    class="message"
    [class.show]="showMessage"
    [ngClass]="{
      success: messageType == 'success',
      error: messageType == 'error'
    }"
  >
    {{ message }}
  </div>
</section>
```

This is an Angular registration form for nurses.

How it works:

- **Form setup:**
  - Uses `formGroup="registerForm"` to bind the form to Angular's reactive form system.
  - Calls `onSubmit()` when the form is submitted.
- **Input fields:**
  - Contains fields for **DNI, Name, Surname, Username, and Password**, each linked to a form control.
  - If a field is invalid and has been touched, an error message appears in red.
- **Submit button:**
  - Disabled if the form is invalid.
- **Message display:**
  - Shows a success or error message when registration is completed.
  - Uses `ngClass` to apply styles based on the message type.

Backend Programming:

```php
#[Route(path: '/Nurses/Create', name: 'app_nurse_create', methods: ['POST'])]
8 references | 0 overrides
public function createNurse(EntityManagerInterface $entityManager, Request $request): JsonResponse
{

    try {

        $entityManager->getRepository(class: Nurses::class)->createNurse(DNI: $request->get(key: "dni"), Nombre: $request->get(key: "name"),
        Apellido: $request->get(key: "surname"), Usuario: $request->get(key: "user"), Password: $request->get(key:…"password"));

        return new JsonResponse(data: "OK", status: Response::HTTP_CREATED);
    } catch (Exception $e) {
        return new JsonResponse(data: "KO", status: Response::HTTP_BAD_REQUEST);
    }
}
```

La línea de createNurses es la misma la de abajo solo que he hecho un enter para que se vea en la imagen. (Está en castellano porque es una cosa que no tendría que decir pero la digo por si acaso).

This PHP function handles nurse registration via a `POST` request.

How it works:

- It listens for requests at `/Nurses/Create`.
- It retrieves `dni`, `name`, `surname`, `user`, and `password` from the request.
- It calls `createNurse()` from the `Nurses` repository to save the new nurse in the database.
- If the operation is successful, it returns a `201 Created (OK)` response.
- If an error occurs, it catches the exception and returns a `400 Bad Request (KO)` response.

Endpoint 'Login'

In this section, the goal is to connect the angular login with the backend endpoint. With this we can log in with the users that are in the database.

Frontend TypeScript Programming:

```
export class Login {
  enfermeros: any[] = [];
  id: any;
  loginForm !: FormGroup;

  constructor(
    private arrnurses: EnfermerosService,
    private router: Router,
    private localStorage: LocalStorageService
  ) {
    this.loginForm = new FormGroup({
      dni: new FormControl('', [Validators.required]),
      pass: new FormControl('', [Validators.required, Validators.minLength(7)]),
    });
  }

  onSubmit() {
    if (this.loginForm?.valid) {
      this.arrnurses
        .getNursesLogin(
          this.loginForm.get('dni')?.value,
          this.loginForm.get('pass')?.value
        )
        .subscribe({
          next: (result) => {
            if (result['status']) {
              this.id = result['id'];
              return this.MessageShowed('Inicio de sesion exitoso', 'success');
            }
          },
          error: (error) => {
            return this.MessageShowed(error['error']['logError'], 'error');
          },
        });
    }
  }
```

In the .ts file, we called the function getNursesLogin located in the injectable when the form is submitted and validated.
Taking the result of the query and seeing if the result ["state"] is true or false. If the query is true, it is because the query went well and the "dni" and "pass" entered match those found in the database. If it does not match, the response will be false.

Also, if the response to the query is true, we will save the "id" of the logged in user through localstorage, so that we can later recover their information in other parts of the website.

Injectable Frontend programming and Backend

```
getNursesLogin(dni: string, pass: string): Observable<any> {
  let url = '/Nurses/Login';

  const params = new HttpParams().set('dni', dni).set('pass', pass);

  const headers = new HttpHeaders({ 'Content-Type': 'application/json' });

  return this.http.post<any>(url, null, { headers, params });
}
```

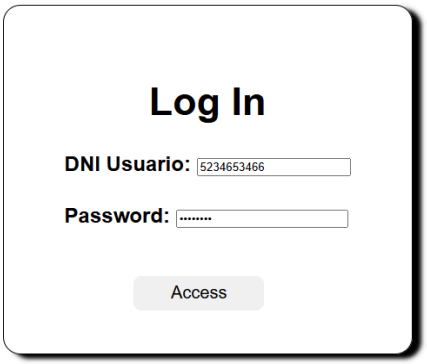
Front End Function

```
#[Route('/Nurses/Login', name: 'app_nurse_login', methods: ['POST'])]

public function nurselogin(EntityManagerInterface $entityManager, Request $request): JsonResponse
{

    try {

        $nurse = $entityManager->getRepository(Nurses::class)->findOneBy(["dni" => $request->get("dni"), "password" =>
        $request->get("pass")]);

        if ($nurse) {
            $id = $nurse->getId();
            return new JsonResponse(["status" => true, "id" => $id, "logError" => null], Response::HTTP_OK);
        }

        throw new Exception(message: "Los datos introducidos no coinciden con ningun usuario existente.");
    } catch (Exception $e) {

        return new JsonResponse(["status" => false, "id" => null, "logError" => $e->getMessage()], Response::HTTP_NOT_FOUND);
    }
}
```

Back End Endpoint

The getNursesLogin() function is responsible for querying the backend using an http request. In this case, the request is of type ["POST"] and sends the "dni" and "pass" as parameters so that in the backend, the comparison of these data with those of the database is carried out.

If this data matches a record it will return true along with the id of the logged in user and a null in the error message, if it does not match any it will return false, a null as id and the error message that appears when executing the query with the BBDD;



```
private MessageShowed(message: string, type: 'success' | 'error'): void {
  this.message = message;
  this.messageType = type;
  this.showMessage = true;

  setTimeout(() => {
    this.showMessage = false;

    if (this.messageType == 'success') {
      this.localStorage.setItem('id', this.id);
      this.router.navigate([]);
    }
  }, 2000);
```

PD: The "id" that the backend query returns if the data entered matches a database user, is subsequently stored in a localstorage, since it is used when wanting to load the user's profile.

HTML Frontend Programming:

```
src > app > login > login.component.html > section > div.login > form.form > div.inputs > div.pass
<section>
  <div class="login">
    <form [formGroup]="loginForm" (ngSubmit)="onSubmit()" class="form">
      <div class="inputs">
        <label class="title"> Log In </label>
        <div class="user">
          <label>
            DNI Usuario:
            <input type="text" formControlName="dni" class="input1" />
          </label>
        </div>
        <div
          *ngIf="
            loginForm?.get('dni')?.invalid && loginForm?.get('dni')?.touched
          "
          style="color: red; font-weight: bold"
        >
          El DNI es obligatorio.
        </div>
        <div class="pass">
          <label>
            Password:
            <input type="password" formControlName="pass" class="input2" />
          </label>
        </div>
        <div
          *ngIf="
            loginForm?.get('pass')?.invalid && loginForm?.get('pass')?.touched
          "
          style="color: red; font-weight: bold"
        >
          Ingresa un password valido.
        </div>
      </div>
      <button type="submit" [disabled]="loginForm.invalid" class="submit">
        Access
      </button>
```

It was not necessary to modify the html of the login part, due to the changes necessary for this part, they only affected the .ts files and the injectable.

Nurse list ‘getAll’

Frontend TypeScript Programming:

This code is for a component in Angular that shows a list of nurses and their images.

- `enfermeros: any = [];`: It creates an empty list called `enfermeros` to store the data of the nurses.
- **Constructor**: The `EnfermerosService` is used to get the data of the nurses from the backend. The service is injected into the component so it can be used.
- `ngOnInit()`: This is a special method that runs when the component is loaded. Inside it, the `getAllNurses()` method from the `EnfermerosService` is called to fetch the nurse data.

- `subscribe()`: This listens for the data:
  - `next()`: If the data is fetched successfully, it's stored in the `enfermeros` array and logged to the console.
  - `error()`: If there's an error while fetching the data, it logs the error to the console.
- `getImagenSrc(i: number): string {}`: This function takes a number (`i`) and returns a path to an image, like `man1.jpg`, `man2.jpg`, etc. This is used to show different images for the nurses.

```
export class ListNursesComponent implements OnInit {
  enfermeros: any = [];
  constructor(private enfermerosService: EnfermerosService) { }

  ngOnInit(): void {

    this.enfermerosService.getAllNurses().subscribe({

      next: (result) => {

        this.enfermeros = result;
        console.log(result)
        console.log(this.enfermeros)
      },
      error: (error) => {
        console.log(error);
      },
    });

  }

  getImagenSrc(i: number): string {
    return `assets/imgs/man${i}.jpg`;
  }
}
```

Injectable Frontend programming:

The function sends a GET request to the `/Nurses/List` URL to get a list of nurses. It expects the response to be a JSON array of `Nurse` objects and sets the `Content-Type` header to `'application/json'`.

```
getAllNurses(): Observable<Array<Nurse>> {
  let url = '/Nurses/List';
  return this.http.get<Array<Nurse>>(url, {
    headers: new HttpHeaders({ 'Content-Type': 'application/json' }),
  });
}
```

HTML Frontend Programming:

This code defines an HTML section that displays a list of nurses. It uses Angular to loop through an array of "enfermeros" (nurses) and creates a table with their information. Each row in the table includes the nurse's NIF, name, surname, username, password, and an image. The `*ngFor` directive is used to repeat the rows for each nurse, and `getImagenSrc(i)` dynamically provides the image source for each nurse.

```html
<section class="container-general">
  <div class="titulo">
    <h1>List of Nurses</h1>
  </div>
  <div class="tabla">
    <table class="tabla-nurses">
      <tr>
        <th>NIF</th>
        <th>Name</th>
        <th>Surname</th>
        <th>User</th>
        <th>Password</th>
        <th>Image</th>
      </tr>
      <tr class="{{ enfermero }}" *ngFor="let enfermero of enfermeros; index as i">
        <td>{{ enfermero["nif"] }}</td>
        <td>{{ enfermero["name"] }}</td>
        <td>{{ enfermero["surname"] }}</td>
        <td>{{ enfermero["user"] }}</td>
        <td>{{ enfermero["pass"] }}</td>
        <td><img [src]="getImagenSrc(i)" alt="No hay Imagen"></td>
      </tr>
    </table>
  </div>
</section>
```

Backend Programming:

Route: The code listens for a GET request at the URL `/Nurses/List`.
Get Nurses: It fetches all the nurses from the database using the `findAll()` method.
Check if nurses exist: If there are any nurses, it loops through each nurse.
Create Response: For each nurse, it creates an array with their NIF, name, surname, username, and password.
Return JSON: It sends back a JSON response with the nurses' data.
Empty Response: If no nurses are found, it returns an empty JSON array.

```php
#[Route(path: '/Nurses/List', name: 'app_nurse_list', methods: ['GET'])]
6 references | 0 overrides
public function list_of_nurses(EntityManagerInterface $entityManager): JsonResponse
{

    $nursesRepository = $entityManager->getRepository(className: Nurses::class);

    $nurses = $nursesRepository->findall();

    if (!empty($nurses)) {
        $result =  [];

        foreach ($nurses as $nurse) {
            $result[] = [
                'nif' => $nurse->getDNI(),
                'name' => $nurse->getName(),
                'surname' => $nurse->getSurname(),
                'user' => $nurse->getUser(),
                'pass' => $nurse->getPassword()
            ];
        }
        return new JsonResponse(data: $result);
    } else {
        return new JsonResponse(data: []);
    }
}
```

Github URL:

URL Frontend: Hospital Frontend
URL Backend: Hospital Backend